

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 1

Roll no: 08 PRN No: 12412097 Name of Student: Sahil Balasaheb Patil

Subject Name : Deep Learning

Problem Statement :

Install and configure TensorFlow/Keras in Google Colab. Perform data preprocessing, normalization, train-test split, and visualization on a sample dataset.

Algorithm :

Step 1: Import Libraries

- Import TensorFlow for building the neural network.
- Import Matplotlib for visualizing images.

Step 2: Load Dataset

- Load the MNIST handwritten digit dataset.
- Split the dataset into:
 - Training set
 - Testing set

Step 3: Preprocess the Data

- Normalize pixel values from 0–255 to 0–1 by dividing every pixel by 255.
- This helps the neural network train faster and improves convergence.

Step 4: Visualize the Dataset

- Display sample images.
- Verify the image size (28×28 pixels).
- Check corresponding labels (digits 0–9).

Step 5: Build the Neural Network

Create a Sequential Neural Network consisting of:

1. Flatten Layer
 - Converts each 28×28 image into a 784-element vector.
2. Hidden Layer
 - Dense layer with 128 neurons.
 - Activation function: ReLU.
3. Output Layer
 - Dense layer with 10 neurons (one for each digit).
 - Activation function: Softmax to produce class probabilities.

Step 6: Compile the Model

Configure the model using:

- Optimizer: Adam
- Loss Function: Sparse Categorical Crossentropy
- Evaluation Metric: Accuracy

Step 7: Train the Model

- Train the neural network using the training dataset.
- Run training for 3 epochs.
- During training:
 - Perform forward propagation.
 - Calculate loss.

- Backpropagate errors.
- Update weights using the Adam optimizer.

Step 8: Evaluate the Model

- Test the trained model on unseen test data.
- Calculate:
 - Test Loss
 - Test Accuracy

Step 9: Save the Model

- Save the trained model for future predictions without retraining.

Code :

```
import tensorflow as tf
import matplotlib.pyplot as plt

# Get data and split into train and test datasets
(train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.mnist.load_data()

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 2s 0us/step

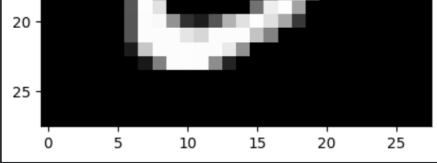
# Scale down values of pixels from 0-255 to 0-1
train_images = train_images / 255.0
test_images = test_images / 255.0

# Visualizing data
print(train_images.shape)
print(test_images.shape)
print(train_labels)

# Display first image
plt.imshow(train_images[1], cmap='gray')
plt.show()
```

... (60000, 28, 28)
(10000, 28, 28)
[5 0 4 ... 5 6 8]





```
# Defining neural network model
my_model = tf.keras.models.Sequential()
my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))
my_model.add(tf.keras.layers.Dense(128, activation='relu')) #Rectified Linear Unit #Dense layers are used to learn complex patter
my_model.add(tf.keras.layers.Dense(10, activation='softmax')) #softmax is often used as the activation function in the output layer for cl

# Compiling the model
my_model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy']) #Adaptive Moment Estimation #sparse_categor

# Training the model
my_model.fit(train_images, train_labels, epochs=3)

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape`/`input_dim` argument
super().__init__(**kwargs)
Epoch 1/3
1875/1875 ----- 9s 4ms/step - accuracy: 0.9261 - loss: 0.2575
Epoch 2/3
1875/1875 ----- 5s 3ms/step - accuracy: 0.9670 - loss: 0.1126
Epoch 3/3
1875/1875 ----- 4s 2ms/step - accuracy: 0.9763 - loss: 0.0780
<keras.src.callbacks.history.History at 0x79e3c44ff500>
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 10)	1,290

Total params: 305,312 (1.16 MB)
Trainable params: 101,770 (397.54 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 203,542 (795.09 KB)

```
# Checking model accuracy on test data
val_loss, val_acc = my_model.evaluate(test_images, test_labels)
print('Test accuracy: ', val_acc)

... 313/313 ----- 2s 5ms/step - accuracy: 0.9761 - loss: 0.0784
Test accuracy: 0.9761000275611877

# Saving the model
my_model.save('my_mnist_model.h5')

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered

Start coding or generate with AI.
```

Output :

```
my_model.summary()

Model: "sequential"

```

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 10)	1,290

```

Total params: 305,312 (1.16 MB)
Trainable params: 101,770 (397.54 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 203,542 (795.09 KB)

# Checking model accuracy on test data
val_loss, val_acc = my_model.evaluate(test_images, test_labels)
print('Test accuracy: ', val_acc)

... 313/313 ----- 2s 5ms/step - accuracy: 0.9761 - loss: 0.0784
Test accuracy: 0.9761000275611877

# Saving the model
```

```
... super().__init__(**kwargs)
Epoch 1/3
1875/1875 ----- 9s 4ms/step - accuracy: 0.9261 - loss: 0.2575
Epoch 2/3
1875/1875 ----- 5s 3ms/step - accuracy: 0.9670 - loss: 0.1126
Epoch 3/3
1875/1875 ----- 4s 2ms/step - accuracy: 0.9763 - loss: 0.0780
<keras.src.callbacks.history.History at 0x79e3c44ff500>

# Checking model accuracy on test data
val_loss, val_acc = my_model.evaluate(test_images, test_labels)
print('Test accuracy: ', val_acc)

313/313 ----- 2s 5ms/step - accuracy: 0.9761 - loss: 0.0784
Test accuracy: 0.9761000275611877

# Saving the model
my_model.save('my_mnist_model.h5')

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered
```